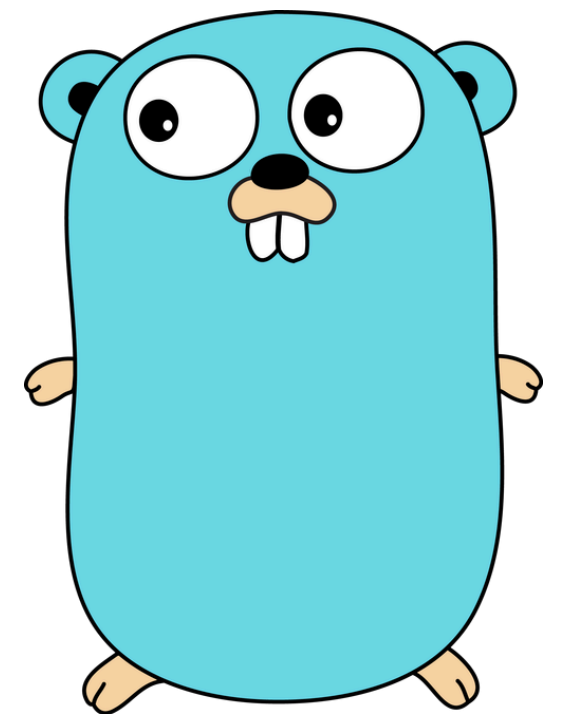


Linguagem Go

WILLIAM, KLAUS

Gopher →



Criação da linguagem

Projetada em 2007 no Google

Criada por Robert Griesemer, Rob Pike e Ken Thompson

**Buscava melhorar a produtividade da
programação**

Anunciada publicamente em 2009

Versão 1.0 lançada em 2012

Características

Open Source

Código simples e legível

Linguagem de alto nível

Linguagem compilada

Compilação estática

Contém garbage collector mark and sweep

Foco em runtime mínimo e concorrência

Tipagem estática com inferência

Aplicações

Aplicações backend

Daemons

Servidores

Aplicativos de terminal

APIs

Hello World



```
package main
```

```
import "fmt"
```

```
func main() {  
    fmt.Println("Hello, World")  
}
```

Go



```
package main

import "fmt"

func main() {
    fmt.Println("Hello, World")
}
```

Java



```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Sintaxe

```
var nome string = "klaus"  
nome := "william"
```

```
nome, idade, raca, fofo := "Lui", 2, "shihtzu", true
```

```
arr[3] = {67, 69, 420}  
arr[...] = {1, 2, 3, 4, 5, 6}
```

```
sli := []int{2, 3, 5, 7, 11, 13}
```

Sintaxe

```
sum := 0
for i := 0; i < 10; i++ {
    sum += i
}
fmt.Println(sum)
```

```
sum := 1
for sum < 1000 {
    sum += sum
}
fmt.Println(sum)
```

Sintaxe

```
var nome string = "klaus"  
nome := "william"
```

```
nome, idade, raca, fofo := "Lui", 2, "shihtzu", true
```

```
arr[3] = {67, 69, 420}  
arr[...] = {1, 2, 3, 4, 5, 6}
```

```
sli := []int{2, 3, 5, 7, 11, 13}
```

```
idades := make(map[string]int)  
idades["Alice"] = 28  
idades["Bob"] = 35  
idades["Charlie"] = 42
```

Sintaxe

```
var nome string = "klaus"  
nome := "william"
```

```
nome, idade, raca, fofo := "Lui", 2, "shihtzu", true
```

```
arr[3] = {67, 69, 420}  
arr[...] = {1, 2, 3, 4, 5, 6}
```

```
sli := []int{2, 3, 5, 7, 11, 13}
```

```
idades := make(map[string]int)  
idades["Alice"] = 28  
idades["Bob"] = 35  
idades["Charlie"] = 42
```

Sintaxe

```
if x > 10 {  
    ...  
}  
else if x < 5 {  
    ...  
}  
else {  
    ...  
}
```

```
if x = func(); x > 10 {  
    ...  
}
```

```
switch x {  
    case 1:  
        ...  
    case 2:  
        ...  
        fallthrough  
    case 3:  
        ...  
    default:  
        ...  
}
```

Boas Práticas

Concorrência

Use a biblioteca padrão

Manter nomes simples e descritivos (net/http, json)

Foco em testes

Propagação de erros

Ferramentas opinionadas (go vet, go fmt)

Convenções

CamelCase

Bracket na mesma linha {

Nomes públicos com letra maiúscula

Nomes curtos

Comentários de linha // não bloco /* */

Nomes exportados devem incluir comentário (geração de docs)

Lidar com errors explicitamente

Convenções

Estrutura de projeto

```
project-root-directory/  
  go.mod  
  internal/  
    auth/  
    ...  
    metrics/  
    ...  
    model/  
    ...  
  cmd/  
    api-server/  
      main.go  
    metrics-analyzer/  
      main.go  
    ...
```

Documentação

go.dev/doc

Why Go ▾Learn**Docs ▾**PackagesCommunity ▾

Documentation

The Go programming language is an open source project to make programmers more productive.

Go is expressive, concise, clean, and efficient. Its concurrency mechanisms make it easy to write programs that get the most out of multicore and networked machines, while its novel type system enables flexible and modular program construction. Go compiles quickly to machine code yet has the convenience of garbage collection and the power of run-time reflection. It's a fast, statically typed, compiled language that feels like a dynamically typed, interpreted language.

Getting Started

Installing Go

Instructions for downloading and installing Go.

Tutorial: Getting started

A brief Hello, World tutorial to get started. Learn a bit about Go code, tools, packages, and modules.

Tutorial: Create a module

A tutorial of short topics introducing functions, error handling, arrays, maps, unit testing, and compiling.

Tutorial: Getting started with multi-module workspaces

Introduces the basics of creating and using multi-module workspaces in Go. Multi-module workspaces are useful for making changes across

Developing modules

Developing and publishing modules

You can collect related packages into modules, then publish the modules for other developers to use. This topic gives an overview of developing and publishing modules.

Module release and versioning workflow

When you develop modules for use by other developers, you can follow a workflow that helps ensure a reliable, consistent experience for developers using the module. This topic describes the high-level steps in that workflow.

Managing module source

When you're developing modules to publish for others to use, you can help ensure that your modules are easier for other developers to use by following the repository conventions described in this topic.

FIM!

